TASKFLOW APP · LIB/

Frontend Guide

Flutter · Riverpod · SQLite · Firebase
Architecture, offline sync, notifications and setup

TaskFlow 1.0.0 · Flutter / Dart 3.13 · Riverpod 3 · go_router · sqflite · dio · FCM

Contents

1. Overview 4
What the app does 4
What changed in this version 4
Main packages 5

2. Getting started 6
Run the app 6
Build settings (dart-defines) 6

3. Architecture 7
Folder structure 7
State management 7
Navigation 8

4. Core building blocks 9
ApiClient (core/network) 9
TokenStore (core/storage) 9
Local database (core/database) 9
DataChangeBus (core/sync) 9
Errors 9
Utilities 10

5. Features 11
Authentication flow 11
Tasks 11
Settings 12

6. Offline-first sync 13
One sync round 13
When it syncs 13
Time zones 14
Status in the UI 14

7. Notifications 15
Local notifications 15
Push notifications 15

8. Firebase and platform setup 16

Firebase project 16

Android 16

iOS (Xcode) 16

Permissions 16

Check that it works 16

9. UI cleanup and performance 17

Removed text and clutter 17

Speed 17

Stability 17

10. Testing and troubleshooting 18

Tests 18

Troubleshooting 18

1. Overview

TaskFlow is an offline-first task manager. Everything is saved on the phone first (SQLite), so the app is instant and works without a connection. When online it syncs with the TaskFlow API, so tasks follow the user to every device, and it receives push notifications through Firebase.

What the app does

- **Accounts** against the API: sign up, log in, "Remember me", automatic session refresh, profile, password change and sign-out.
- **Tasks** with notes, subtasks, priority, category, due date and time or all-day, reminders and repeat rules (daily, weekdays, weekly, monthly).
- **Quick add** that understands the title as you type: `Call Sam tomorrow at 5pm #personal !high`.
- **Views**: Home (today, upcoming, overdue), Search with filters, Calendar month view, Stats (flow score, on-time rate, streaks, categories) and Profile.
- **Sync** across devices in the background, with an offline indicator and a manual sync row on the profile screen.
- **Notifications**: local reminders and a morning digest scheduled on the device, plus Firebase push for changes made elsewhere.
- **Backup**: export and import all tasks as JSON through the clipboard.

What changed in this version

Area	Change
Backend	The app now talks to the new Node.js / Neon API (see the Backend guide). Sign-in is real; passwords are no longer stored on the phone.
Sync	New `sync` feature: pushes local edits, pulls remote ones, resolves conflicts, adopts tasks created before the account existed.
Push	New `PushService` (Firebase Cloud Messaging) and `devices` feature that registers the install and its push token.
Data	SQLite schema v5: stable UUIDs for tasks and subtasks, sync state, tombstones. Broken and duplicate task / settings data classes were removed and rebuilt cleanly.
UI	Filler copy removed, lighter widgets (no glow, shadows or pulsing animations), lazy slivers for long lists, optimistic updates. See chapter 8.
Quality	Dead files removed, new unit tests for DTOs, settings sync, UUIDs and API errors, CI workflow for app and server.

Main packages

Package	Used for
`flutter_riverpod 3`	State management and dependency injection (providers).
`go_router 18`	Navigation, the tab shell and the signed-in / signed-out route guard.
`sqflite`	The local database: tasks and subtasks.
`shared_preferences`	Settings, the cached profile, sync cursors.
`flutter_secure_storage 10`	Access and refresh tokens (Keychain on iOS, Keystore on Android).
`dio 5`	HTTP client with the auth interceptor (token refresh and retry).
`firebase_core, firebase_messaging`	Remote push notifications.
`flutter_local_notifications, timezone, flutter_timezone`	Scheduled reminders and digest at the right local time.
`intl`	Date and number formatting.

2. Getting started

Run the backend first (Backend guide, chapter 2), then the app.

Run the app

Terminal

```
flutter pub get
flutter run                                          # default API URL (see below)
flutter run --dart-define-from-file=config/dev.json  # your own settings

flutter analyze
flutter test
```

`pubspec.lock` is not included; `flutter pub get` creates it with the newest compatible versions. Commit it afterwards so every build uses the same versions.

Build settings (dart-defines)

Settings are compiled in with `--dart-define` or, easier, a JSON file passed with `--dart-define-from-file`. Copy `config/example.json` to `config/dev.json` (and `config/prod.json`); the `config/*.json` files are git-ignored except the example.

config/example.json

```
{
  "API_BASE_URL": "https://your-api.example.com/api/v1",
  "FIREBASE_API_KEY": "",
  "FIREBASE_PROJECT_ID": "",
  "FIREBASE_MESSAGING_SENDER_ID": "",
  "FIREBASE_ANDROID_APP_ID": "",
  "FIREBASE_IOS_APP_ID": "",
  "IOS_BUNDLE_ID": "com.taskflow.taskflow"
}
```

Where the app runs	API_BASE_URL
Android emulator (default)	`http://10.0.2.2:8080/api/v1` (the emulator's alias for your computer)
iOS simulator, desktop (default)	`http://localhost:8080/api/v1`
Real phone on the same Wi-Fi	`http://<your computer's LAN IP>:8080/api/v1` (debug builds only)
Release builds	An HTTPS URL, e.g. `https://api.yourdomain.com/api/v1`

Note. Plain HTTP is allowed only in debug builds (Android `usesCleartextTraffic` in the debug manifest; iOS `NSAllowsLocalNetworking`). Release builds must use HTTPS.

Without the Firebase values the app runs normally, without remote pushes: local reminders, sync and everything else keep working.

3. Architecture

Feature-first clean architecture: each feature has `data`, `domain` and `presentation` layers, and shared infrastructure lives in `core`. The backend follows the same structure, so both halves read the same way.

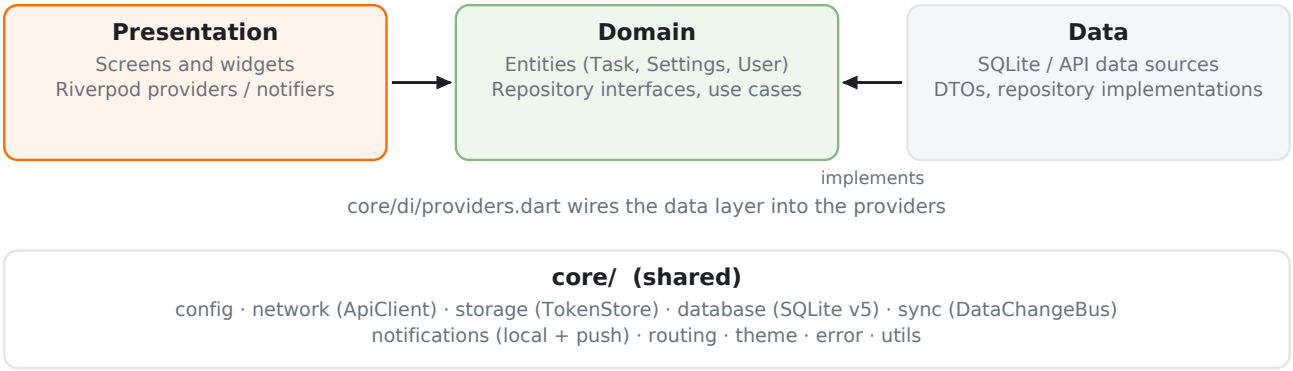


Figure 1. Layers of a feature. Arrows show the direction of dependencies.

Folder structure

lib/

```
lib/
  main.dart                start-up: bindings, notifications, Firebase, ProviderScope
  app.dart                 MaterialApp.router; starts sync, reminders, device registration
  core/
    config/app_config.dart  dart-define settings (API URL, Firebase options)
    network/api_client.dart dio client: auth header, X-Device-Id, token refresh, errors
    storage/token_store.dart  tokens in secure storage (or memory without "Remember me")
    database/app_database.dart  SQLite schema v5 and upgrades
    sync/data_change_bus.dart   decouples features from the sync engine
    notifications/          local notifications, push service, reminder rescheduling
    routing/                go_router, route guard, splash gate
    theme/, error/, di/, utils/  (quick add parser, dates, uuid, time zone)
  features/
    auth/       data (local + remote), domain (User, repository, use cases), presentation
    tasks/      data (SQLite source, DTOs, repository, backup), domain, presentation
    sync/       data (remote source, SyncEngine), domain (SyncStatus), presentation
    devices/    data (remote source), presentation (registration provider)
    settings/   data (DTO, repository), domain (Settings, repository), presentation
    stats/      domain (productivity maths), presentation (stats screen)
  shared/widgets/           app shell (dock), header, UI kit, progress ring
```

State management

Riverpod providers form the dependency graph. Infrastructure is provided once in `core/di/providers.dart`; features read it with `ref.watch` and never construct it themselves, which keeps them testable (tests override the providers).

Provider	Holds
`authProvider`	The signed-in `User` (async). Restores the session at start-up and signs out when the server ends the session.

Provider	Holds
`taskListProvider`	All tasks of the user plus filter and search state; optimistic updates.
`settingsProvider`	Preferences; every change is saved locally and marked for upload.
`syncEngineProvider / syncStatusProvider`	The engine for the signed-in user and its status (idle, syncing, offline, error, last synced).
`syncControllerProvider`	Decides when to sync (see chapter 5).
`reminderSyncProvider`	Reschedules local notifications when tasks or settings change.
`deviceRegistrationProvider`	Registers the install and its push token with the API.
`statsReportProvider`	The stats report per period, computed once and cached.
`apiClientProvider, tokenStoreProvider, installationIdProvider`	Shared infrastructure.

Navigation

Route	Screen
`/splash`	Start-up while the session is restored.
`/login, /signup`	Authentication (only when signed out).
`/home, /search, /calendar, /stats, /profile`	The five tabs inside the app shell and dock.
`/add-task, /edit-task/:id`	Create or edit a task.
`/task/:id`	Task details with subtasks.

The route guard sends signed-out users to `/login` and signed-in users away from the auth screens. Tapping a notification opens `/task/:id` for the task it belongs to.

4. Core building blocks

Shared pieces every feature builds on.

ApiClient (core/network)

- A dio client with the base URL from `AppConfig`, timeouts (10 s connect, 20 s send, 30 s receive), `Authorization: Bearer ...` and `X-Device-Id` on every request.
- A `QueuedInterceptor` handles 401s: the first failing request refreshes the tokens once, others wait and then retry with the new token. If the refresh is rejected, `onSessionExpired` fires and the user is signed out.
- Errors become `Failure`s: `NetworkFailure` (offline, timeouts) or `ApiFailure` with the server's `detail` and field errors, so screens can show them as they are.

TokenStore (core/storage)

Keeps the access and refresh tokens in `flutter_secure_storage` (Keychain / Keystore). With "Remember me" off the tokens live in memory only, so closing the app signs the user out. The refresh token is rotated by the server on every refresh.

Local database (core/database)

SQLite database `taskflow.db`, schema version 5. The upgrade from older versions keeps every task: it adds the new columns, gives existing tasks and subtasks a UUID and creates the indexes.

Table	Columns (sync-related in bold)
tasks	id (local integer), **uuid** (shared with the server), **userId**, title, description, dueDate, dueTime, priority, category, isCompleted, isAllDay, reminder, reminderMinutes, repeatRule, completedAt, createdAt, **updatedAt**, **syncState** (0 synced, 1 pending), **deletedAt** (tombstone)
subtasks	id, **uuid**, taskId, title, isCompleted, **position**
users	Legacy offline accounts from earlier versions, used only to adopt their tasks after the first sign-in.

DataChangeBus (core/sync)

A small event hub that keeps features independent of the sync engine. Repositories call `localChanged` after a write; the sync controller listens and schedules a push. The engine calls `remoteChanged` after a pull; the task and settings providers listen and reload. `syncNow()` lets any screen (e.g. pull-to-refresh) run a sync and wait for it.

Errors

`core/error/failures.dart` defines `Failure` and its subclasses. `describeError()` turns any exception (dio, SQLite, format) into a short, user-facing sentence, so no raw exception text reaches the UI.

Utilities

- `uuid.dart`: RFC 4122 v4 UUIDs from a secure random source (ids are created offline).
- `device_timezone.dart`: the device's IANA zone (e.g. `Asia/Colombo`), sent with tasks and settings so the server computes reminders and digests at the right time.
- `quick_add_parser.dart`: reads dates, times, `#category` and `!priority` from a title.
- `date_time_utils.dart`: formatting and parsing of times and dates.

5. Features

Each feature owns its screens, state and data. They talk to each other only through providers and the change bus.

Feature	Responsibility	Key files
auth	Sign up, log in, session restore and expiry, profile refresh, sign out.	`auth_remote_data_source`, `auth_local_data_source` (cached profile), `auth_repository_impl`, `auth_provider`
tasks	Task list, search, calendar, details, add / edit, completion and repeat, backup.	`task_local_data_source`, `task_model` (row + DTO mapping), `task_repository_impl`, `task_provider`
sync	Push local changes, pull remote ones, report status.	`sync_engine`, `sync_remote_data_source`, `sync_provider`
devices	Register this install (platform, push token, time zone, app version).	`device_remote_data_source`, `device_registration_provider`
settings	Preferences, synced with last-write-wins; the profile screen.	`settings_repository_impl`, `settings_dto`, `settings_provider`, `profile_screen`
stats	Productivity numbers and the stats screen.	`productivity.dart`, `stats_screen`

Authentication flow

1. On start the splash screen restores the session: tokens from secure storage and the cached profile, so the app opens instantly, even offline. The profile is refreshed from `/users/me` in the background.
2. Log in / sign up call the API, save the tokens (persisted only with "Remember me"), cache the user and navigate home. Sign-up sends the device time zone.
3. While signed in, expired access tokens are refreshed automatically. If the server ends the session (password changed elsewhere, token revoked), the user is signed out.
4. Sign-out unregisters the device and ends the session on the server (skipped when offline), then clears the tokens and the cached profile. Local tasks stay on the phone for when the user signs in again.

Tasks

- All reads and writes go to SQLite, scoped to the signed-in user. Every write marks the row `pending` and stamps `updatedAt`; deletes set `deletedAt` so the deletion can sync.
- The list is updated **optimistically**: a checkbox or edit shows at once and the database catches up right after.
- Completing a repeating task creates the next occurrence once (tapping again cannot stack copies).
- Limits match the server: subtask titles up to 200 characters and up to 50 subtasks per task; longer titles and notes are trimmed to the API limits before upload.
- Pull-to-refresh runs a sync and reloads; offline, the local list is simply shown.

Settings

Settings live in shared preferences with an `updatedAt` stamp and a "dirty" flag. They are uploaded with the next sync and the newest edit wins across devices. The first sync after sign-up uploads the settings the phone already had. If a different account signs in on the same phone, the previous account's unsent edits are not uploaded to the new one.

6. Offline-first sync

The phone is the source of truth for the screens; the server is the meeting point between devices. `SyncEngine` moves changes in both directions.

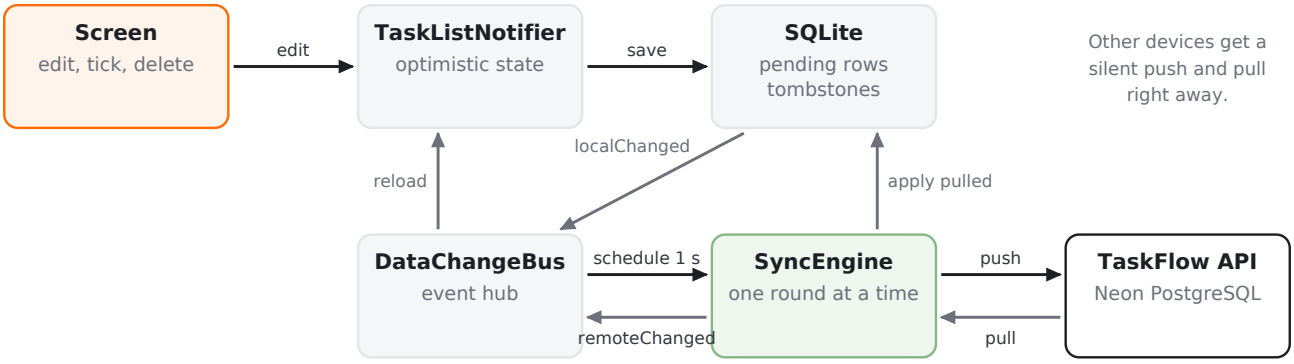


Figure 2. An edit is saved locally first; the engine pushes it and pulls changes from other devices.

One sync round

1. **Adopt** (first round after sign-in): tasks created before the account existed, or by an old offline account with the same email, are given to this user and marked pending.
2. **Push**: pending tasks and tombstones go up in batches of 100 as whole tasks (with subtasks), together with the settings if they changed. Each result (`applied`, `stale`, `rejected`) marks the row as sent; newer server versions arrive with the pull.
3. **Pull**: changes after the saved cursor, page by page, until `hasMore` is false. Each task is applied unless the local copy has newer unsent edits. Tombstones delete locally.
4. **Finish**: save the cursor and the "last synced" time, emit the status, notify the screens through `remoteChanged`.

Only one round runs at a time. Asking again while one runs queues exactly one more round, so a burst of edits never causes a burst of requests.

When it syncs

Trigger	Delay
After sign-in or start-up with a saved session	immediately
After any local change (via the change bus)	1 second, debounced
App returns to the foreground	immediately
While the app is open	every 5 minutes
Silent "sync" push from the server (another device changed something)	immediately
Pull-to-refresh	immediately, and waits for the result

Time zones

Tasks are sent as calendar day + time + IANA zone. A timed task created in Colombo at 2 PM shows as 9:30 AM on a phone set to London (same instant); all-day tasks keep their calendar day everywhere. If the zone cannot be read, the push is skipped for that round (the pull still runs) and retried next time.

Status in the UI

- A small offline icon in the header when the last sync failed for lack of a connection.
- A sync row on the profile screen: last synced time, errors, and a button to sync now.

7. Notifications

Two complementary paths: local notifications scheduled on the phone (they work offline and are exact), and Firebase push from the server for changes this phone has not seen yet.

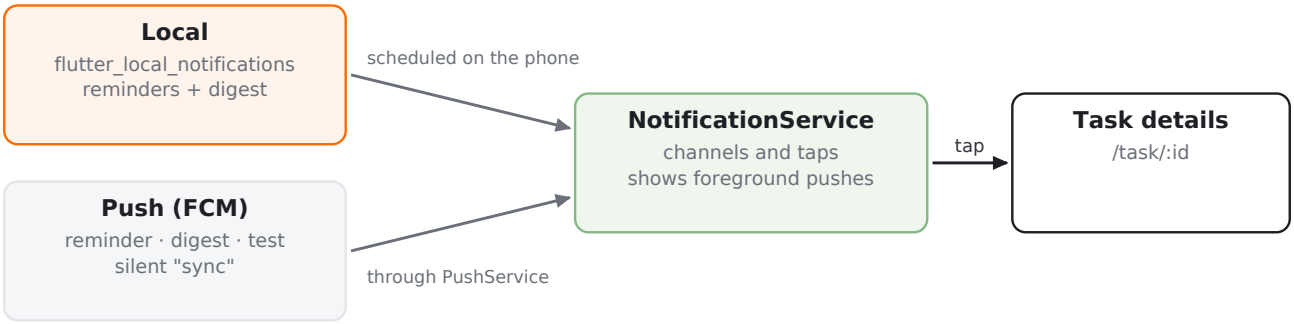


Figure 3. Both paths end in the same channels; a tap opens the task.

Local notifications

- Channel `task_reminders` (high importance): one reminder per open task with a reminder, `reminderMinutes` before it is due (all-day tasks at 9 AM).
- Channel `daily_digest`: the morning summary at the time chosen in Profile.
- `reminderSyncProvider` reschedules everything 400 ms after tasks, settings or the session change, so a batch of edits causes one reschedule. Signing out cancels them.
- Scheduled with the device time zone, so they fire correctly across DST changes. `RECEIVE_BOOT_COMPLETED` restores them after a reboot on Android.

Push notifications

Push type	App behaviour
`sync (silent)`	Starts a sync right away; the pulled changes also reschedule local reminders.
`reminder`	Sent only when this phone does not know the latest version of the task yet. In the background the OS shows it; in the foreground Android shows it through the local channel (iOS shows it natively). It also triggers a sync.
`digest`	Only for devices that rely on the server for the digest.
`test`	From `POST /notifications/test`, to check the setup.

`PushService` initialises Firebase from the dart-define values before `runApp`. It exposes streams for sync requests, notification taps and token changes. `deviceRegistrationProvider` sends the token, platform, app version and time zone to `PUT /devices/{installationId}` after sign-in and whenever the token changes.

8. Firebase and platform setup

The app initialises Firebase from build settings, so no `google-services.json` or `GoogleService-Info.plist` needs to be added.

Firebase project

1. Create a project at **console.firebase.google.com**.
2. Add an **Android** app with package `com.taskflow.taskflow` and an **iOS** app with bundle id `com.taskflow.taskflow`.
3. From **Project settings > General > Your apps** copy the API key, project id, sender id and the two app ids into `config/dev.json`.
4. Give the server a service account key (Backend guide, chapter 10).

Android

- Already configured: `INTERNET`, `POST_NOTIFICATIONS`, `RECEIVE_BOOT_COMPLETED`, the default FCM channel (`task_reminders`), notification icon and accent colour.
- Debug builds allow plain HTTP for a local API; release builds need HTTPS.

iOS (Xcode)

1. Open `ios/Runner.xcworkspace`, select **Runner > Signing & Capabilities**.
2. Add **Push Notifications** and **Background Modes** with **Remote notifications** checked (`UIBackgroundModes` is already in `Info.plist`).
3. Create an APNs authentication key at developer.apple.com and upload it in Firebase (**Project settings > Cloud Messaging**).
4. Push notifications need a real iPhone; the simulator does not receive them.

Permissions

- The notification permission (Android 13+ `POST_NOTIFICATIONS`, iOS alerts) is asked for when the user turns on a reminder for a task or enables notifications in Profile.
- On iOS, asking for a push token after sign-in shows the same system prompt if it has not been answered yet.
- If the user declines, the app keeps working; only the notifications are missing.

Check that it works

1. Run the backend with `FIREBASE_SERVICE_ACCOUNT` set, and the app with your config file.
2. Sign in on the phone and allow notifications.
3. Call `POST /api/v1/notifications/test` with the user's token (Swagger UI at `/docs`), or edit a task on a second device and watch the first one update.

9. UI cleanup and performance

The screens were trimmed to what the user needs, and the widget tree was made lighter so scrolling and navigation stay smooth.

Removed text and clutter

Screen	Change
Splash, login, sign-up	Shorter copy; the privacy card and extra taglines were removed.
Home	Focus card removed; the list starts with what is due.
Add / edit task	Mode pill, "mindful" banner and the mood priority removed; priority is Low / Medium / High, matching the server.
Stats	Explanatory pill and tooltips removed.
Profile	Trimmed copy; new sync row.
Empty states, dialogs, 404	Secondary messages are optional and omitted where the title says enough.

Speed

- Home and Search use slivers, so only visible task cards are built.
- `RepaintBoundary` around the dock, each task card and the splash backdrop: animations and scrolling no longer repaint unrelated parts of the screen.
- No transparent shadows, opacity layers, glow or pulsing animations (expensive to draw on every frame).
- Route transitions are a short 220 ms fade.
- Stats are cached per period (`statsReportProvider`) instead of being recomputed on every build.
- Optimistic updates: taps respond before the database write finishes.
- Debounced work: reminders reschedule once per burst (400 ms), sync once per burst (1 s).

Stability

- Broken and duplicate task and settings data classes were replaced by one clean implementation per layer; dead files and unused dependencies were removed.
- Swiping away a subtask can no longer act twice on the same item.
- Input limits match the server (text lengths, subtask count), so uploads are never rejected for size; long text is trimmed safely without breaking emoji.
- A load started for a previous account is discarded when another account signs in.
- Errors from the network, database and parsing are shown as short messages instead of crashing or showing raw exceptions.

10. Testing and troubleshooting

Tests

File	Covers
`task_dto_test`	JSON mapping with the API, time zones, all-day tasks, text limits.
`settings_sync_test`	Dirty flag, last-write-wins, account switching.
`api_error_test`	Turning API and network errors into messages.
`uuid_test`	UUID format and uniqueness.
`auth_restore_test, route_guard_test`	Session restore and redirects.
`task_list_state_test, repeat_rule_test`	Filters, sorting, repeat rules.
`quick_add_parser_test, date_time_utils_test`	Quick add and date helpers.
`productivity_test, task_backup_test, password_rules_test`	Stats, backup, password rules.

Terminal

```
flutter test                # all tests
flutter analyze             # lints (flutter_lints)
```

Important. This version was prepared without a Flutter SDK available, so it was reviewed carefully but not compiled here. Run `flutter pub get`, `flutter analyze` and `flutter test` once after unpacking; the CI workflow does the same on every push.

Troubleshooting

Symptom	Fix
"Can't reach the server" on the emulator	Is the API running? Use `10.0.2.2` (Android emulator) or `localhost` (iOS simulator).
Works on emulator, not on a real phone	Use the computer's LAN IP in `API_BASE_URL` and allow port 8080 in the firewall, or use an HTTPS URL.
Release build cannot connect	Release builds require HTTPS.
Signed out unexpectedly	The session was ended on the server (password change or revoked token). Sign in again.
No push notifications	Firebase values missing in the config file, notifications denied, or the server has no service account. Try the test push.
No push on iOS	Push capability and APNs key are required; use a real device.
Tasks missing on another device	Check the sync row in Profile; pull to refresh.
Build errors after unpacking	Run `flutter pub get` (the lock file is regenerated), then `flutter clean` if needed.